

ORCHARD DOCUMENT 2

Security, Safety, and Apple Silicon Acceleration Requirements

Enterprise Agent Execution Manual Addendum

This document is the second strict manual for agent-led Orchard development. The first manual covers learning, architecture, repo scaffolding, reconciliation, Apple container internals, and documentation discipline. This addendum covers the two areas that must be formalized before serious implementation: security/safety and Apple silicon acceleration research.

The rules in this document are written as execution constraints for local AI coding agents, research agents, and review agents. Agents may study, scaffold, document, and propose, but they must not make destructive runtime, security, installer, cleanup, or accelerator claims without evidence and tests.

Version wording such as MVP/v2/v3 is intentionally absent. The project language is: supported release, release track, capability set, engineering gate, and acceptance criteria.

1. Document Purpose

This document adds security and acceleration requirements to the Orchard engineering baseline. It defines how agents must research Apple silicon acceleration surfaces, how they must handle security and destructive operations, and what repository artifacts must exist before implementation begins.

The key correction is practical: performance and safety are not afterthoughts. The first supported release must be built with measurable resource behavior, clear runtime boundaries, safe defaults, explicit permissions, and a source-backed understanding of Apple silicon acceleration.

1.1 Source Anchors

Agents must use primary sources first. Secondary analysis may be used only to generate questions, not to establish project commitments.

ID	Source	Why it matters
S1	Apple container technical overview	Defines one-VM-per-container runtime architecture, OCI behavior, vmnet, XPC, launchd, Keychain, unified logging, container-apiserver, helpers, memory ballooning limitation, and macOS 15 network limitations.
S2	Apple Containerization repository	Defines Swift package surface, Apple silicon/macOS 26/Xcode 26 requirements, vminitd, fast-boot kernel, Rosetta support, lightweight VM design, and runtime process APIs.
S3	Apple Metal overview	Defines Metal as low-overhead GPU compute/graphics API for Apple silicon, including ML integration, MPS, MPSGraph, Core ML model integration, PyTorch backends, Instruments profiling, and Metal 4 support on Apple silicon Macs.
S4	Apple MLX repository	Defines MLX as Apple silicon ML array framework with CPU/GPU execution, lazy computation, dynamic graphs, Swift/C/C++/Python APIs, and unified memory behavior.
S5	SLSA official site	Defines supply-chain integrity framework, controls, provenance, and progressive assurance levels.
S6	Apple Keychain / XPC / launchd / platform security docs	Defines native security primitives to study for credentials, helper boundaries, daemon behavior, and permission prompts.

2. Non-Negotiable Agent Rules

Rule	Instruction
Evidence before claim	Every claim about GPU, Neural Engine, Metal, Core ML, MLX, virtualization, Keychain, XPC, launchd, registry auth, or container runtime behavior must cite a source or be

	marked as unverified.
No private APIs	Agents must not propose private ANE, undocumented accelerator, kernel, hypervisor, or security bypass routes for shipping behavior.
No silent destructive behavior	Any operation that can delete, overwrite, detach, unmount, stop, restart, expose, or reconfigure resources must have a dry-run model and an audit event.
No broad compatibility promises	Agents must not promise GPU/ANE/Metal availability inside Linux container VMs until it is verified through official documentation or reproducible experiments.
No root by default	Agents must assume user-space execution unless a specific capability requires a helper. Any helper requires a design note, threat model, and privilege minimization plan.
No secret leakage	Secrets must not appear in manifests, logs, events, status output, crash reports, test fixtures, screenshots, or documentation examples.
No unbounded scheduling	Agents must model memory, CPU, architecture, thermal, and battery impact before adding workload management behavior.
No implementation without gate	Security-sensitive and acceleration-sensitive code requires design docs, tests, review checklist, rollback plan, and failure-mode analysis.

3. Security and Safety Scope

Security covers how Orchard protects users, host machines, project data, credentials, runtime resources, and supply-chain integrity. Safety covers operational correctness: avoiding accidental deletion, avoiding unrecoverable partial state, avoiding hidden privilege escalation, and making failures understandable.

3.1 Threat Model

Threat area	Questions agents must answer
Local user mistakes	Can a normal command accidentally delete named volumes, host files, project networks, or long-lived images? What preview does the user see?
Malicious manifest	Can a shared manifest mount sensitive host paths, expose ports, consume excessive memory, use wrong-architecture images, or inject secrets into logs?
Compromised image	Can an image abuse host mounts, credentials, network access, or project-level permissions? How are images pinned, scanned, signed, or restricted?
Secret exposure	Can registry tokens, application secrets, local env files, Keychain items, logs, events, crash dumps, or support bundles leak data?
Runtime drift	Can out-of-band runtime changes confuse reconciliation into deleting or recreating the wrong resource?
Privilege boundary	Can a privileged helper be avoided? If not, what exact operations require it and how is the helper constrained?
Supply chain	Can build artifacts, release binaries, installation scripts,

	schemas, and docs be tampered with or replaced?
Network exposure	Can Orchard unintentionally bind public interfaces, bypass project isolation, or expose services beyond localhost?

3.2 Safety Requirements

ID	Requirement	Acceptance gate
SAFE-001	Every apply operation must support a dry-run plan that lists create, update, stop, delete, network, volume, and config actions.	A reviewer can inspect the exact action plan before mutation.
SAFE-002	Named volumes must not be deleted by default.	down/clean commands preserve named volumes unless an explicit destructive flag and confirmation are provided.
SAFE-003	Orphan cleanup must distinguish Orchard-owned resources from user-owned resources.	Resources include ownership labels/metadata and cleanup refuses ambiguous deletion.
SAFE-004	Partial apply failure must leave a recoverable operation record.	The operation ledger shows completed, pending, failed, and rollback-blocked actions.
SAFE-005	Restart policies must include crash-loop backoff.	A failing service does not restart indefinitely at high frequency.
SAFE-006	Host mounts must be least-privilege and path-scoped.	Mounting home directory, Downloads, Desktop, SSH keys, browser profiles, Keychains, or broad root paths triggers a hard validation block or explicit policy override.
SAFE-007	Secrets must be redacted from all status, logs, events, and reports.	Automated tests inject known secret patterns and verify redaction.
SAFE-008	Every destructive command must emit an audit event.	Events include actor, command, project, resource, action, result, timestamp, and error.
SAFE-009	Commands that affect networking must show bind address and exposure risk.	Public bind attempts require explicit confirmation or policy approval.
SAFE-010	Resource pressure checks must run before scheduling.	Apply fails or warns when declared workload budget exceeds configured safety limits.

3.3 Security Requirements

ID	Requirement	Acceptance gate
SEC-001	Define a written threat model before security-sensitive implementation.	docs/architecture/security-model.md exists and is reviewed.
SEC-002	Registry credentials must use native credential storage where available.	Credential paths are documented; no tokens are stored in plaintext Orchard state.
SEC-003	The daemon must run with the smallest practical privilege set.	User-space design is documented; privileged helper use is justified per

		operation.
SEC-004	All release artifacts must have provenance and checksums.	Release pipeline publishes checksums and provenance metadata.
SEC-005	Image identity controls must support digest pinning.	Manifest schema supports image digests or a policy that blocks mutable tags in strict mode.
SEC-006	Source control must include security automation.	CI includes dependency audit, secret scanning, static checks, and a security scorecard target.
SEC-007	Installer scripts must be minimal, auditable, and non-destructive.	Install/uninstall scripts have dry-run or documented file impact.
SEC-008	Logs must be structured and privacy-aware.	Unified logging / local logs avoid secrets and include privacy levels where applicable.
SEC-009	Network and mount policies must be enforceable from config.	Policy files can block sensitive mounts, public ports, amd64 images, and unpinned tags.
SEC-010	Security review is required for cleanup, registry auth, Keychain, XPC, launchd, installer, update, and host-service code.	PR template includes security impact section and reviewer checklist.

3.4 Security Repository Additions

docs/architecture/security-model.md

docs/architecture/safety-model.md

docs/architecture/permissions-and-helpers.md

docs/reference/destructive-operations.md

docs/reference/secret-handling.md

docs/reference/policy.md

docs/reference/audit-events.md

docs/design/adr-0011-safety-gates.md

docs/design/adr-0012-user-space-first.md

docs/design/adr-0013-secret-redaction.md

docs/design/adr-0014-release-provenance.md

security/threat-model.md

security/release-checklist.md

security/reporting.md

Security and Safety Boundary Map

Every boundary crossing needs explicit ownership, permission, validation, and audit events.

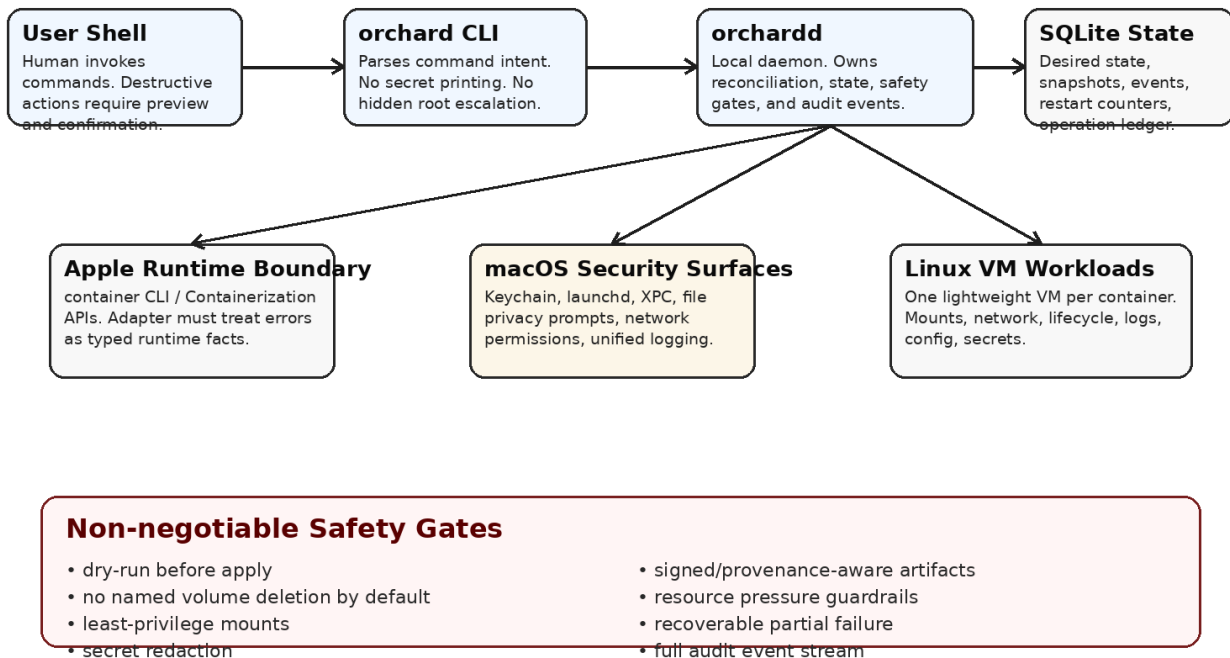


Figure 1. Security and safety boundary map.

4. Apple Silicon Acceleration Scope

This section defines the research and design constraints for GPU, Neural Engine, Metal, Core ML, MLX, PyTorch MPS, and unified-memory-aware workloads. The core instruction is to measure and verify. A supported behavior must be backed by official documentation or repeatable experiments.

4.1 Acceleration Research Questions

Area	Required questions
Linux VM boundary	Can workloads inside Apple container lightweight Linux VMs access any Apple GPU, Metal, Core ML, ANE, or MLX acceleration path? What is officially supported? What is impossible? What is merely unverified?
Metal	What Metal APIs are accessible only to host-native macOS processes? What profiling tools are available through Instruments, Metal System Trace, performance HUD, API validation, and shader validation?
Core ML / ANE	Which accelerator routing is available through Core ML? What can run on CPU/GPU/ANE? What can be configured or observed? What must remain host-native?
MLX	How does MLX use CPU/GPU and unified memory? What workloads and examples matter for local inference, embeddings, fine-tuning, and model serving?
PyTorch MPS	Which local workflows use MPS? What are memory and compatibility caveats? How does it compare with MLX and llama.cpp for local use?
Rosetta / amd64	What happens when images run as linux/amd64 on Apple silicon? What performance, correctness, and energy risks follow?
Resource policy	How should workload profiles express CPU, memory, architecture, accelerator, thermal, and battery policy?
Host-native services	When acceleration is host-native, how should Orchard supervise host processes and connect them to containerized services?

4.2 Acceleration Requirements

ID	Requirement	Acceptance gate
ACC-001	The system must record host chip family, memory, macOS version, Apple container version, and architecture compatibility during doctor checks.	orchard doctor prints a machine capability report.
ACC-002	The manifest must support architecture policy for container workloads.	Strict mode can prefer or require linux/arm64 images.
ACC-003	linux/amd64 images on Apple silicon must produce a visible compatibility warning or require policy approval in strict profiles.	A test manifest using amd64 triggers the expected diagnostic.
ACC-004	The project must maintain an accelerator capability matrix.	docs/reference/accelerator-compatibility.md lists host-native and

		container-VM support states.
ACC-005	No GPU/ANE/Metal/Core ML claim may be documented as supported until verified.	Unverified capabilities remain marked research-only.
ACC-006	AI-heavy workload examples must include memory budget, thermal notes, architecture notes, and safety limits.	Examples include resource profiles and failure behavior.
ACC-007	Host-native acceleration services require explicit supervision semantics.	If adopted, hostServices have lifecycle, logs, env, secrets, restart, and cleanup rules.
ACC-008	Acceleration testing must include idle, cold start, warm start, sustained load, sleep/wake, and memory pressure scenarios.	Bench reports exist under docs/benchmarks or perf/.
ACC-009	Resource profiles must be enforceable.	Profiles can block over-budget applies and warn about battery/thermal risk.
ACC-010	Profiling methodology must be reproducible.	Scripts document Instruments, powermetrics, memory_pressure, vm_stat, and runtime logs where applicable.

4.3 Supported-State Vocabulary

Agents must classify every acceleration capability using this vocabulary. Do not use stronger wording than the evidence supports.

State	Meaning
supported	Backed by official documentation and an Orchard test or experiment.
experimentally verified	Observed in a repeatable local experiment, but not established as official support.
host-native only	Available through macOS-native process surfaces, not through Apple container Linux VM workloads.
container-available	Available from inside Apple container workloads through a documented path.
blocked	Known unsupported or incompatible.
unknown	Not enough evidence yet.

4.4 Apple Silicon Optimization Requirements

ID	Requirement
SIL-001	Prefer arm64 images and record architecture at pull/create time.
SIL-002	Measure VM boot latency for cold and warm starts.
SIL-003	Measure idle daemon CPU and memory.
SIL-004	Measure container VM memory high-water behavior under load.
SIL-005	Measure energy and thermal behavior under sustained

	workloads.
SIL-006	Handle laptop sleep/wake with explicit reconciliation policy.
SIL-007	Avoid busy polling; prefer event-driven observation where available, backoff otherwise.
SIL-008	Maintain benchmark fixtures for single-service, multi-service, AI-adjacent, and memory-heavy stacks.

Apple Silicon Acceleration Research Map

Treat GPU, ANE, unified memory, and Linux VM boundaries as measurable system facts.

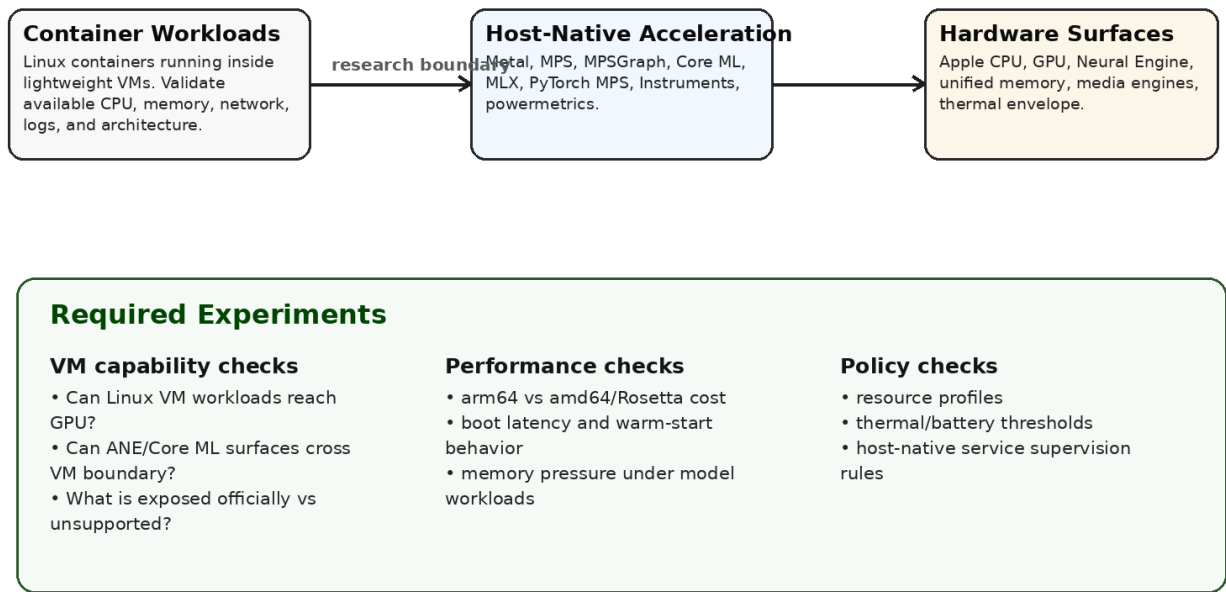


Figure 2. Apple silicon acceleration research map.

5. Host-Native Service Research Track

Some Apple silicon acceleration surfaces may require macOS-native processes rather than Linux VM containers. Agents must research this boundary carefully and design a supervised host-service model only if the evidence justifies it.

5.1 Required Host-Service Questions

1. Can a host-native service be supervised without weakening Orchard safety guarantees?
2. How are host-service logs captured and redacted?
3. How are host-service env vars and secrets injected without plaintext leakage?
4. How are ports assigned and connected to container workloads?
5. How are crash-loop backoff and cleanup handled?
6. How are launchd, process groups, signals, and child process cleanup handled?
7. How does a host-native service declare memory and accelerator expectations?
8. How does sleep/wake affect host-native services compared with lightweight VM containers?

9. What security review is required before allowing hostServices in supported releases?

5.2 Draft Manifest Surface for Research Only

```
project:
  name: local-rag-stack

services:
  api:
    image: ghcr.io/example/rag-api:sha256-...
    dependsOn:
      - postgres
      - local-llm

  postgres:
    image: postgres:16@sha256:...
    volumes:
      - notes-db:/var/lib/postgresql/data

hostServices:
  local-llm:
    command: ["mlx-server", "--model", "qwen"]
    resources:
      memory: 12Gi
      accelerator: apple-gpu
    restartPolicy: unlessStopped
    exposure:
      bind: 127.0.0.1
      port: 11434
```

This surface is not an implementation commitment. It is a research artifact that forces agents to answer lifecycle, security, logging, secret, port, resource, and cleanup questions before code exists.

6. Policy Engine Requirements

Security and acceleration rules need enforcement. A policy system can start as static validation and later become configurable enterprise policy.

Policy	Default posture
architecture	Prefer linux/arm64. Warn or block linux/amd64 in strict profiles.
image tags	Allow mutable tags in development profile; block unpinned images in strict/shared profiles.
public ports	Require explicit declaration and confirmation.

host mounts	Block sensitive paths unless explicit policy allows them.
secrets	Disallow plaintext secrets in shared manifests.
resource budget	Block applies that exceed configured safe memory budget.
cleanup	Preserve named volumes by default.
acceleration	Require capability classification before documenting support.

6.1 Policy File Sketch

```
policy:
  profile: strict
  images:
    requireDigest: true
    allowAmd64OnAppleSilicon: false
  mounts:
    deny:
      - ~/Library/Keychains
      - ~/.ssh
      - ~/Library/Application Support/*/Profiles
      - /
  networking:
    requireExplicitPublicBind: true
  secrets:
    allowPlaintextInManifest: false
  resources:
    maxContainerMemoryPercent: 60
    requireMemoryBudget: true
```

7. Agent Execution Workflow for Document 2

Agents must follow this workflow before adding security-sensitive or acceleration-sensitive code.

Stage	Agent output
1. Source collection	Source log with primary links, exact claims, and unknowns.
2. Capability matrix	Tables for security surfaces, macOS permissions, runtime capabilities, GPU/ANE/Metal/Core ML/MLX access, and Linux VM boundary.
3. Threat model	Threat actors, assets, trust boundaries, abuse cases, safety hazards, and controls.
4. Requirements	SHALL-style requirements with acceptance gates and tests.
5. Experiments	Reproducible scripts for accelerator capability checks, resource measurements, and safety edge cases.

6. Design note	Architecture decision with tradeoffs and explicit unsupported/unknown areas.
7. Tests	Unit, integration, policy, redaction, destructive-operation, sleep/wake, and benchmark tests.
8. Review packet	Security impact, privacy impact, performance impact, rollback plan, and docs impact.

8. Required Experiments

8.1 Security Experiments

Experiment	Pass condition
Secret redaction	Known secret values never appear in status, logs, events, errors, support bundles, or test snapshots.
Mount policy	Sensitive paths are blocked under strict policy and require explicit override under development policy.
Named volume preservation	down/clean preserves named volumes by default.
Partial apply failure	Injected failures produce recoverable operation records and do not delete unrelated resources.
Registry credential path	Credentials are not written into Orchard SQLite, logs, or manifests.
Public port warning	Public bind addresses produce explicit warnings and policy behavior.

8.2 Acceleration and Performance Experiments

Experiment	Metrics
arm64 vs amd64 image behavior	startup time, CPU use, memory use, warning quality, Rosetta path if applicable.
Container VM acceleration capability	official support classification plus experiment result for GPU/Metal/Core ML/ANE visibility inside Linux VM.
Host-native MLX service	start/stop latency, logs, memory, GPU use, crash behavior, port binding, cleanup.
Sustained local model workload	tokens/sec or task throughput, memory pressure, thermals, battery drain, UI responsiveness.
Sleep/wake behavior	service survival, stale sockets, reconciliation after wake, event correctness.
Daemon idle behavior	idle CPU, idle memory, number of wakeups, polling interval impact.

9. Documentation Additions

docs/architecture/apple-silicon-acceleration.md

docs/architecture/accelerator-boundaries.md

docs/architecture/security-model.md
docs/architecture/safety-model.md
docs/architecture/permissions-and-helpers.md
docs/architecture/host-native-services.md
docs/reference/accelerator-compatibility.md
docs/reference/resource-policy.md
docs/reference/destructive-operations.md
docs/reference/secret-handling.md
docs/reference/policy.md
docs/reference/audit-events.md
docs/benchmarks/apple-silicon-baseline.md
docs/design/adr-0011-safety-gates.md
docs/design/adr-0012-user-space-first.md
docs/design/adr-0013-secret-redaction.md
docs/design/adr-0014-release-provenance.md
docs/design/adr-0015-apple-silicon-acceleration-research.md
docs/design/adr-0016-host-native-services-research.md

10. Strict Prompt for Local Agents

Use this prompt as a second research mission after the existing Orchard engineering manual.

You are working on Orchard Document 2: Security, Safety, and Apple Silicon Acceleration Requirements.

Your job is to research and formalize security, operational safety, and Apple silicon acceleration constraints before implementation.

Rules:

- Do not implement runtime, cleanup, installer, security, policy, credential, host-service, or accelerator code.
- Do not claim GPU, Neural Engine, Metal, Core ML, or MLX support inside Apple container workloads unless verified by official documentation or repeatable experiment.
- Treat private or undocumented accelerator paths as research-only and blocked for supported release behavior.
- Produce requirements with IDs, acceptance gates, tests, and repo/doc artifacts.
- Every destructive action must have dry-run, confirmation policy, ownership checks, and audit events.
- Every secret path must have redaction tests.
- Every acceleration claim must be classified: supported, experimentally verified, host-native only, container-available, blocked, or unknown.

Deliverables:

1. Source log with primary sources and claim mapping.
2. Security threat model.
3. Safety requirements table.
4. Apple silicon acceleration capability matrix.
5. Host-native service research note.
6. Policy engine requirements.
7. Required experiments and benchmark plan.
8. Repository additions.
9. ADR drafts.
10. Open questions that block implementation.

Review standard:

If evidence is missing, mark the item unknown. Do not fill gaps with confident guesses.

11. Review Checklist

Area	Reviewer must confirm
Security model	Assets, actors, trust boundaries, abuse cases, and controls are documented.
Destructive operations	Dry-run, confirmation, ownership, and audit events exist.
Secrets	No plaintext secret path leaks into state, logs, events, docs, tests, or screenshots.
Supply chain	Release plan includes checksums, provenance, dependency audit, and signed artifact research.
macOS permissions	Keychain, file access, network, XPC, launchd, and helper boundaries are documented.
Acceleration	Capabilities are classified and not overstated.
Resource behavior	Memory, CPU, battery, thermal, sleep/wake, and idle behavior have experiments.
Host services	Lifecycle, logs, cleanup, secrets, ports, resources, and security are defined before implementation.
Docs	Every supported behavior has docs; every unsupported behavior has explicit error text.

12. Open Questions That Block Implementation

10. Can Apple container Linux VM workloads access Apple GPU acceleration through any supported path?
11. Can Apple container Linux VM workloads access ANE/Core ML through any supported path?
12. Which acceleration surfaces must remain host-native?
13. What is the exact behavior of linux/amd64 containers using Rosetta under Apple container?
14. What runtime metadata exposes image architecture, image digest, memory limits, exits, logs, and VM lifecycle?

15. Can registry credentials be accessed through Apple container/Keychain without duplicating secrets in Orchard state?
16. What daemon model gives low idle overhead without missing drift events?
17. What user-space operations require macOS privacy prompts?
18. What network operations require permission or packet filter changes?
19. How does sleep/wake affect container-apiserver, helpers, VMs, networks, and host-native services?
20. What is the minimum support bundle needed for debugging without leaking secrets?
21. What strict profile rules should apply to team-shared manifests?
22. How should partial apply rollback work when Apple container actions are not transactional?
23. What benchmarks define acceptable performance for the first supported release?
24. What official Apple roadmap changes would invalidate the current security or acceleration assumptions?

13. References

- [1] **Apple container technical overview:** <https://github.com/apple/container/blob/main/docs/technical-overview.md>
- [2] **Apple Containerization repository:** <https://github.com/apple/containerization>
- [3] **Apple Metal overview:** <https://developer.apple.com/metal/>
- [4] **MLX repository:** <https://github.com/ml-explore/mlx>
- [5] **SLSA official site:** <https://slsa.dev/>
- [6] **Apple Keychain Services:** <https://developer.apple.com/documentation/security/keychain-services>
- [7] **Apple XPC documentation:** <https://developer.apple.com/documentation/xpc>
- [8] **Apple Virtualization framework:** <https://developer.apple.com/documentation/virtualization>
- [9] **Kubernetes security concepts:** <https://kubernetes.io/docs/concepts/security/>
- [10] **Kubernetes controllers:** <https://kubernetes.io/docs/concepts/architecture/controller/>

End of Document 2.